Tarea 1 - Trabajando con hash

Mauricio Torres

Apartado 1. Cálculo de hashes

Algoritmo	Longitud
MD5	32 caracteres
SHA-1	40 caracteres
SHA-256	64 caracteres
SHA-512	128 caracteres
`b2sum`	128 caracteres

Cambio de un carácter

Hash original:

```
openssl dgst -sha256 mensaje.txt
SHA2-256(mensaje.txt)= 3b3c1a1d6319561bd1ebf8c214cd77833e6
```

Después de cambiar el carácter `s` de «hamburguesa»:

```
openssl dgst -sha256 mensaje.txt
SHA2-256(mensaje.txt)= 6ae8c5e8c5286c171e09017e4ce1a7a4400ec13881b3e9124f91f7fbaea547f4
```

Compara ambos hashes: ¿Cuántos caracteres coinciden en la misma posición? ¿Qué propiedad demuestra esto?

Ningún carácter está en la misma posición; todo el hash ha cambiado. Esto demuestra que el mínimo cambio de un carácter cambia por completo el hash.

`echo` y `echo -n`

5. Genera el hash de la misma cadena con `echo` y con `echo -n`. ¿Por qué son diferentes?

Son diferentes porque `echo` añade un carácter de nueva línea al final de la cadena. En cambio, `echo -n` no lo añade.

Preguntas

a) ¿Es posible «descifrar» un hash?

No se puede descifrar porque el hash generado no se puede revertir, pero sí se pueden realizar ataques de fuerza bruta para intentar saber el texto original del archivo.

b) ¿Qué es una colisión y por qué MD5 y SHA-1 se consideran inseguros?

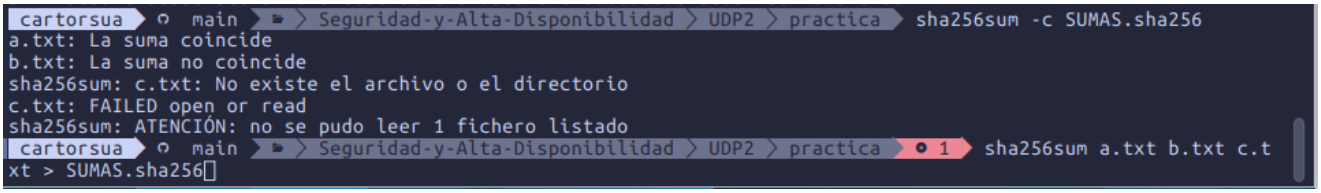
MD5 y SHA-1 se consideran inseguros porque se han encontrado métodos prácticos para generar colisiones:

- **MD5:** es vulnerable desde hace años; se pueden crear archivos distintos con el mismo hash.
- **SHA-1:** también tiene colisiones demostradas.

Apartado 2. Verificación de archivos

5. Verificación después de borrar `c.txt`

```
 sha256sum -c SUMAS.sha256
a.txt: La suma coincide
b.txt: La suma no coincide
sha256sum: c.txt: No existe el archivo o el directorio
c.txt: FAILED open or read
sha256sum: ATENCIÓN: no se pudo leer 1 fichero listado
```



En el anterior apartado dice que el hash no coincide y ahora es que directamente no encuentra el archivo.

6. Opción `--ignore-missing`

Comprueba cómo se utilizaría la opción `--ignore-missing` y explica en qué situación real es útil.

Es útil cuando tienes varios archivos que quieres verificar y sus hashes han cambiado, para que no salgan los errores de los archivos que no están y no aparezcan tantas veces por terminal.

Apartado 3. Verificación de una descarga real

Preguntas

a) ¿Por qué no basta con comprobar el hash si lo has descargado del mismo servidor que el fichero?

Si un atacante logra comprometer el servidor de descargas, tiene la capacidad de modificar tanto la imagen `.iso` como el fichero de sumas (`SHA256SUMS`) para que ambos coincidan con la versión maliciosa introducida.

Al calcular el hash localmente y compararlo con el fichero de sumas descargado del mismo servidor alterado, el sistema dará por buena una ISO manipulada.

b) Diferencia entre integridad y autenticidad

Integridad: propiedad que asegura que los datos no han sido alterados, corrompidos o modificados, ni accidental ni maliciosamente, durante su almacenamiento o transmisión.

Autenticidad: propiedad que valida la identidad del creador o emisor del fichero, garantizando que proviene de una fuente legítima y confiable.

Comprobación del hash

```
cartorsua  ~  Descargas  sha256sum -c --ignore-missing Rocky-10.2-x86_64-minimal.iso C
CHECKSUM         CORRECCION.pdf
cartorsua  ~  Descargas  sha256sum -c --ignore-missing Rocky-10.2-x86_64-minimal.iso CHECKSUM
sha256sum: Rocky-10.2-x86_64-minimal.iso: no properly formatted checksum lines found
Rocky-10.2-x86_64-minimal.iso: La suma coincide
cartorsua  ~  Descargas  1  sha256sum -c Rocky-10.2-x86_64-minimal.iso CHECKSUM
sha256sum: Rocky-10.2-x86_64-minimal.iso: no properly formatted checksum lines found
sha256sum: Rocky-10.2-x86_64-boot.iso: No existe el archivo o el directorio
Rocky-10.2-x86_64-boot.iso: FAILED open or read
sha256sum: Rocky-10.2-x86_64-dvd1.iso: No existe el archivo o el directorio
Rocky-10.2-x86_64-dvd1.iso: FAILED open or read
Rocky-10.2-x86_64-minimal.iso: La suma coincide
sha256sum: Rocky-10-latest-x86_64-boot.iso: No existe el archivo o el directorio
Rocky-10-latest-x86_64-boot.iso: FAILED open or read
sha256sum: Rocky-10-latest-x86_64-dvd.iso: No existe el archivo o el directorio
Rocky-10-latest-x86_64-dvd.iso: FAILED open or read
sha256sum: Rocky-10-latest-x86_64-minimal.iso: No existe el archivo o el directorio
Rocky-10-latest-x86_64-minimal.iso: FAILED open or read
sha256sum: ATENCIÓN: no se pudieron leer 5 ficheros listados
```

Apartado 4. Verificación de paquetes

3. Descarga y comprobación del paquete

Descarga el paquete sin instalarlo con `apt download` y calcula su SHA-256. ¿Coincide con el que indica el repositorio?

Sí, coincide con la del repositorio.

```
carlos@carlos-VirtualBox:~$ apt-cache show nginx
Package: nginx
Priority: optional
Section: web
Installed-Size: 1593
Maintainer: Ubuntu Developers <ubuntu-devel-discuss@lists.ubuntu.com>
Architecture: amd64
Version: 1.28.3-2ubuntu1.12
Replaces: nginx-core (<< 1.22.1-6~), nginx-extras (<< 1.22.1-6~), nginx-light (<< 1.22.1-6~)
Provides: httpd, httpd-cgi, nginx-abi-1.28.3-1
Depends: libc6 (>= 2.34), libcrypt1 (>= 1:4.1.0), libpcre2-8-0 (>= 10.22), libssl3t64 (>= 3.0.0), zlib1g (>= 1
:1.1.4), nginx-common (= 1.28.3-2ubuntu1.12)
Breaks: nginx-core (<< 1.22.1-6~), nginx-extras (<< 1.22.1-6~), nginx-light (<< 1.22.1-6~)
Filename: pool/main/n/nginx/nginx_1.28.3-2ubuntu1.12_amd64.deb
Size: 617742
MD5sum: 5f498da10fa617b07c9571023854e686
SHA1: 1209780d65d6a02d68344b45bcb0fc13a2490f13
SHA256: 190e602e4b54cf629c6f625ae1513241db404311c2af98720695b210a6dbe86b
SHA512: 0a72554f542f48c2d47eb3fb912ba9ed9a8014000d550c8be985ad06da2416618baf35a7cdd8215d46325d82d45ab2537a3d2f
6516631754b0f2add0fde209da
Homepage: https://nginx.org
Description-en: small, powerful, scalable web/proxy server
 Nginx ("engine X") is a high-performance web and reverse proxy server
 created by Igor Sysoev. It can be used both as a standalone web server
 and as a proxy to reduce the load on back-end HTTP or mail servers.
Description-md5: cb534cf82475d3e706f730549c99229f
Original-Maintainer: Debian Nginx Maintainers <pkg-nginx-maintainers@alioth-lists.debian.net>
```

```
carlos@carlos-VirtualBox:~$ sha256sum nginx_1.28.3-2ubuntu1.12_amd64.deb
190e602e4b54cf629c6f625ae1513241db404311c2af98720695b210a6dbe86b  nginx_1.28.3-2ubuntu1.12_amd64.deb
```

Apartado 4.2. Verificación de paquetes instalados

```
carlos@carlos-VirtualBox:~$ sudo apt update | grep InRelease | head

WARNING: apt does not have a stable CLI interface. Use with caution in scripts.

Obj:1 http://es.archive.ubuntu.com/ubuntu resolute InRelease
Obj:2 http://es.archive.ubuntu.com/ubuntu resolute-updates InRelease
Obj:3 http://es.archive.ubuntu.com/ubuntu resolute-backports InRelease
Obj:4 http://security.ubuntu.com/ubuntu resolute-security InRelease
Obj:5 https://repo.mongodb.org/apt/ubuntu jammy/mongodb-org/6.0 InRelease
carlos@carlos-VirtualBox:~$
```

Apartado 3

Esto indica que el archivo instalado difiere del original del paquete. La copia de seguridad conserva el binario correcto y permite restaurarlo.

```
carlos@carlos-VirtualBox:~$ cp /usr/bin/tree /usr/bin/tree.bk
cp: no se puede crear el fichero regular '/usr/bin/tree.bk': Permiso denegado
carlos@carlos-VirtualBox:~$ sudo cp /usr/bin/tree /usr/bin/tree.bk
carlos@carlos-VirtualBox:~$ printf 'X'| dd of=/usr/bin/tree bs=1 seek=1000 conv=notrunc
dd: fallo al abrir '/usr/bin/tree': Permission denied
carlos@carlos-VirtualBox:~$ sudo printf 'X'| dd of=/usr/bin/tree bs=1 seek=1000 conv=notrunc
dd: fallo al abrir '/usr/bin/tree': Permission denied
carlos@carlos-VirtualBox:~$ printf 'X'| sudo dd of=/usr/bin/tree bs=1 seek=1000 conv=notrunc
1+0 registros en
1+0 registros fuera
1 byte copiado, 0.000078864 s, 1.0 kB/s
carlos@carlos-VirtualBox:~$ debsums tree
/usr/bin/tree                                                    FAILED
/usr/share/doc/tree/README.gz                                        OK
/usr/share/doc/tree/TODO                                             OK
/usr/share/doc/tree/changelog.Debian.gz                              OK
/usr/share/doc/tree/copyright                                        OK
/usr/share/man/man1/tree.1.gz                                        OK
carlos@carlos-VirtualBox:~$ dpkg --verify tree
??5??????   /usr/bin/tree
carlos@carlos-VirtualBox:~$
```

```
carlos@carlos-VirtualBox:~$ debsums tree
/usr/bin/tree                                                        OK
/usr/share/doc/tree/README.gz                                        OK
/usr/share/doc/tree/TODO                                             OK
/usr/share/doc/tree/changelog.Debian.gz                              OK
/usr/share/doc/tree/copyright                                        OK
/usr/share/man/man1/tree.1.gz                                        OK
carlos@carlos-VirtualBox:~$
```

```
carlos@carlos-VirtualBox:~$ debsums tree
/usr/bin/tree                                                        OK
/usr/share/doc/tree/README.gz                                        OK
/usr/share/doc/tree/TODO                                             OK
/usr/share/doc/tree/changelog.Debian.gz                              OK
/usr/share/doc/tree/copyright                                        OK
/usr/share/man/man1/tree.1.gz                                        OK
carlos@carlos-VirtualBox:~$
```

Preguntas

a) ¿Por qué usar `[trusted=yes]` en un repositorio es una mala práctica de seguridad?

Desactiva la verificación de autenticidad y firma del repositorio. Un atacante podría introducir paquetes maliciosos mediante un repositorio comprometido o un ataque *Man-in-the-Middle*, sin que APT los rechace.

b) ¿Qué limitación tiene `debsums` frente a un atacante que ya tiene acceso root?

No protege frente a un atacante con acceso `root`: puede modificar los ficheros instalados y también las bases de datos, sumas de comprobación o incluso el propio programa `debsums`. Por tanto, sus resultados podrían falsificarse o evitarse.

Apartado 5. Hashes de contraseñas en Linux

Información del usuario

```
username: alumno_hash
contraseña: $y$j9T$C6Lp2nDv2pgf41.3aqrCf1$EmdMqCeOcLCPvAIGEuAylHkezVwecMnSkv3rjKmOPRD
fecha del último cambio de contraseña: 20766
edad mínima de contraseña: 0
edad máxima de contraseña: 99999
periodo de aviso de caducidad: 7
```

```
carlos@carlos-VirtualBox:~$ sudo cat /etc/shadow |  grep alumno_hash
alumno_hash:$y$j9T$C6Lp2nDv2pgf41.3aqrCf1$EmdMqCeOcLCPvAIGEuAylHkezVwecMnSkv3rjKmOPRD:20766:0:99999:7:::
alumno_hash2:$y$j9T$mHxSzxYJTmHGjs/whD5Ao0$iQhUQAtJeAgadWHrPhUtaOqCQtp22KLyn6mSxBIzfG5:20766:0:99999:7:::
carlos@carlos-VirtualBox:~$
```

```
$y$ --> algoritmo es el yescrypt
$j9T$ --> parámetros de configuración
$C6Lp2nDv2pgf41.3aqrCf1$ --> salt
$EmdMqCeOcLCPvAIGEuAylHkezVwecMnSkv3rjKmOPRD$ --> hash resultante
```

Hash de la misma contraseña

```
carlos@carlos-VirtualBox:~$ openssl passwd -6 Hash2026!
$6$sH4PgcUdR6G.K3Ro$VYOv5Z5HuY6BKammBTOa5BaHkA1Ks1yjhLRhu5xF2ubc3dEHfBy0W1K4tEXQU8L2SlBFCWyQpp1Kx0RrbG5TA.
carlos@carlos-VirtualBox:~$ openssl passwd -6 Hash2026!
$6$Oz6cpaAvlZl4KLgi$MReyJuK656bJyCYEIZomqsQDT.7anBXR6D7QF2eHQUJzxJXB211Mtp23k9BNG3yzOwjYBts2SSKnRbioqJIU8.
carlos@carlos-VirtualBox:~$
```

Los hashes nunca serán iguales porque cada vez se generará uno nuevo por seguridad, para que no haya dos hashes iguales.

```
alumno_hash@carlos-VirtualBox:/home/carlos$ openssl passwd Hash2026!
$1$KCGrtzy0$pYDGOGWhycm67BIJghL8k1
alumno_hash@carlos-VirtualBox:/home/carlos$ openssl passwd Hash2026!
$1$JjN1zkAd$5A0m4ZnvluGlJIQKrl/1O/
alumno_hash@carlos-VirtualBox:/home/carlos$
```

Apartado 4

```
carlos@carlos-VirtualBox:~$ openssl passwd -salt C6Lp2nDv2pgf41.3aqrCf1 -6 Hash2026!
$6$C6Lp2nDv2pgf41.3$ORzEZNzf1AjdKLsh74/wdTiOddFMCKnEEX/cD4IZqWwaUDupv2PG0F0iM89XMMlyyvykPKQlKTo9KQdP/D1mH.
```

Sí, obtiene la misma salt que el usuario `alumno_hash`.

Apartado 5

Solo los dos primeros parámetros son iguales; todo lo demás es diferente al primer usuario. Lo que no cambia son las fechas de mínimo y máximo y el cambio de contraseña.

Preguntas

a) ¿Por qué se usan algoritmos como yescrypt o bcrypt en lugar de SHA-256 para contraseñas?

Es una tabla de consultas precalculadas que asigna valores hash a sus entradas de texto plano originales.

b) ¿Qué es una *rainbow table* y cómo lo neutraliza la salt?

Una tabla *rainbow* es una tabla de consulta precalculada que asigna valores hash a sus entradas de texto plano originales, en lugar de calcular el hash de cada contraseña posible durante un ataque.

La defensa es el salteo: agregar un valor aleatorio a cada contraseña antes de aplicarle el hash.